

MDS-007 — Tile Framework

External publication draft

MDS-007 — Tile Framework

A Tile is not a component. It is the physical expression of an Expression.

Purpose

Every previous specification has progressively transformed the user's World into runtime understanding.

The MDL established:

- Vision
- Principles
- Mental Model
- Interaction
- Composition

The MDS established:

- Design Tokens
- Colour
- Materials
- Typography
- Motion
- Composition Engine

MDS-007 defines the final abstraction before components.

It explains how solved runtime Expressions become reusable presentation primitives called **Tiles**.

Unlike traditional UI frameworks, which begin with reusable widgets, Mosaic begins with behavioural Expressions.

Tiles are the bridge between runtime understanding and visible interface.

Relationship to Previous Specifications

[text]

Vision

↓

Principles

↓

Mental Model

↓

Interaction

↓

Composition

↓

Tokens

↓

Colour

↓

Materials

↓

Typography

↓

Motion

↓

Composition Engine

↓

Tile Framework

↓

Component Library

↓

Presentation

The Tile Framework consumes:

- Expressions
- Runtime Hierarchy
- Material Intent
- Typography Intent
- Motion Intent

It produces reusable presentation primitives.

Scope

This specification defines:

- Tile Philosophy
- Tile Taxonomy
- Expression-to-Tile Mapping
- Tile Lifecycle
- Adaptive Tile Behaviour
- Tile Composition
- Tile Interaction
- Runtime Tile Resolution
- Extension Tile Integration
- Tile Orchestration

This specification intentionally does **not** define:

- concrete UI components
- platform widgets
- rendering engines
- layout systems

Those belong to MDS-008 and platform implementations.

Core Question

MDS-007 exists to answer one question.

How should solved runtime Expressions become reusable interface primitives?

Not:

Which widgets should we render?

Tile Statement

Within Mosaic:

Tiles communicate understanding. Components merely render Tiles.

This distinction allows behaviour to remain independent from implementation.

Tile Responsibilities

The Tile Framework separates runtime presentation into several conceptual layers.

[text]
Expression

↓

Tile

↓

Component

↓

Rendering

Each layer contributes one responsibility.

No layer duplicates another.

Expected Outcome

After reading MDS-007 contributors should understand:

- what a Tile is
- why Tiles exist
- how Expressions become Tiles
- how Tiles adapt across devices
- how Tiles participate in runtime behaviour
- how extensions naturally inherit presentation

without discussing specific UI frameworks.

Repository Structure

[text]
design/

■■■ mds/

■■■ MDS-007 Tile Framework/

README.md

00-document-control.md

01-tile-philosophy.md

02-tile-taxonomy.md

03-expression-mapping.md

04-tile-lifecycle.md

05-adaptive-tiles.md

06-tile-composition.md

07-tile-interaction.md

08-runtime-tile-resolution.md

09-extension-tiles.md

10-tile-orchestration.md

11-governance.md

12-adrs.md

13-contributor-guidance.md

glossary.md

references.md

Dependencies

Required reading:

- MDL-001 → MDL-005
- MDS-001 → MDS-006

Downstream specifications:

- MDS-008 Component Library

Review Status

Status

Draft

Owner

Lead Runtime Presentation Architecture Team

Next File

00-document-control.md

Document Control

Document Information

Property	Value
Document ID	MDS-007
Title	Mosaic Design System — Tile Framework
Classification	Internal
Status	Draft
Version	0.1
Owner	Lead Runtime Presentation Architecture Team
Parent Specifications	MDL-001 → MDL-005, MDS-001 → MDS-006
Repository	/design/mds/MDS-007 Tile Framework/

Purpose

MDS-007 defines the Tile Framework used throughout Mosaic.

The Tile Framework is the architectural bridge between runtime understanding and visual implementation.

Where the Composition Engine determines:

What the user should understand.

The Tile Framework determines:

How that understanding becomes reusable presentation.

Tiles are not components.

They are presentation primitives that carry behavioural meaning independently from rendering technology.

Every visual interface within Mosaic should ultimately be composed from Tiles.

Authority

MDS-007 governs:

- Tile philosophy
- Tile taxonomy
- Expression-to-Tile mapping
- Tile lifecycle
- Adaptive Tile behaviour
- Tile interaction
- Runtime Tile resolution

- Tile orchestration
- Extension Tile integration

This specification intentionally does **not** govern:

- platform widgets
- rendering frameworks
- UI toolkits
- animation APIs
- layout engines

Those systems implement Tiles.

They do not define them.

Relationship To MDS

The Tile Framework extends the Composition Engine.

[mermaid]
flowchart TD

```

Vision
Vision --> Principles
Principles --> MentalModel
MentalModel --> Interaction
Interaction --> Composition
Composition --> Tokens
Tokens --> Colour
Colour --> Materials
Materials --> Typography
Typography --> Motion
Motion --> CompositionEngine
CompositionEngine --> TileFramework
TileFramework --> ComponentLibrary
ComponentLibrary --> Presentation

```

The Tile Framework consumes:

- Expressions
- Runtime Hierarchy
- Material Intent
- Typography Intent
- Motion Intent

It produces:

- reusable presentation primitives.

Design Intent

Traditional interface frameworks frequently begin with reusable components.

Examples include:

- cards
- buttons
- lists
- grids

Mosaic intentionally introduces an additional abstraction.

[text]
Expression

↓

Tile

↓

Component

↓

Rendering

This separation ensures that behavioural meaning remains independent from implementation.

Reader Expectations

Before reading this specification contributors should already understand:

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model
- MDS-001 Design Token Architecture
- MDS-002 Colour System
- MDS-003 Material System
- MDS-004 Typography System
- MDS-005 Motion System
- MDS-006 Composition Engine

The Tile Framework assumes runtime understanding has already been solved.

Its responsibility is reusable presentation.

Architectural Scope

The Tile Framework defines:

- tile identities

- tile behaviour
- tile adaptation
- tile interaction
- tile orchestration
- runtime tile resolution

It intentionally avoids implementation technologies such as:

- Flutter widgets
- React components
- SwiftUI views
- Compose composables

Those become consumers of Tiles rather than architectural concepts.

Stability

Expected lifetime.

Artefact	Expected Lifetime
Components	Months
Rendering Frameworks	Months
Platform Widgets	Months
Tile Taxonomy	Years
Tile Philosophy	Decades

Rendering technologies are expected to evolve rapidly.

Tile identities should remain remarkably stable.

Success Criteria

MDS-007 succeeds when:

- Expressions consistently resolve into appropriate Tiles
- Tiles remain reusable across every Mosaic client
- components become simple rendering implementations
- extensions naturally inherit platform presentation
- adaptive behaviour remains predictable
- contributors think in Tiles rather than widgets

Users should never perceive Tiles directly.

They should simply experience a coherent interface whose behaviour naturally reflects the Runtime World.

Review Status

Status

Draft

Dependencies

- MDL-001 → MDL-005
- MDS-001 → MDS-006

Supersedes

None.

Next File

01-tile-philosophy.md

Tile Philosophy

Purpose

Before defining Tile taxonomy, lifecycle or runtime behaviour, contributors must first understand what a Tile represents within Mosaic.

Most interface frameworks begin with reusable components.

Examples include:

- cards,
- lists,
- buttons,
- tables.

These are implementation concepts.

Mosaic intentionally begins one layer higher.

A Tile is **not** an implementation.

It is a behavioural presentation primitive.

It exists to communicate one solved Expression from the Composition Engine.

Philosophy Statement

A Tile is the physical expression of one behavioural Expression within the user's current World.

Everything within the Tile Framework derives from this statement.

Why Tiles Exist

The Composition Engine produces:

- Hero
- Timeline
- Relationships
- Metadata
- Actions
- Progress

These are Expressions.

Rendering frameworks cannot directly render Expressions.

Likewise...

Components should never understand runtime behaviour.

Tiles exist between those worlds.

[text]
Expression

↓

Tile

↓

Component

↓

Rendering

This separation preserves one of the strongest architectural boundaries within Mosaic.

Tiles Are Behavioural

A Tile communicates behaviour.

Not implementation.

Example.

Incorrect.

[text]
PostCard

Correct.

[text]
Hero Tile

The Hero Tile may become:

- a card,
- a full-width panel,
- a television Hero,
- a compact phone presentation.

Its behavioural identity remains unchanged.

Tiles Are Physical

Unlike Expressions, Tiles possess physical identity.

They inherit:

- Materials
- Typography
- Motion

- Interaction

A Tile therefore becomes the first runtime artefact that users can perceive.

Expressions remain conceptual.

Tiles become tangible.

Tiles Are Reusable

A Timeline remains:

Timeline

regardless of:

- media type,
- device,
- platform,
- layout.

Likewise.

A Timeline Tile should remain:

Timeline Tile

Its presentation evolves.

Its identity remains stable.

Tiles Are Presentation Independent

Tiles intentionally avoid:

- Flutter widgets
- React components
- SwiftUI views
- Compose composables

Those belong to implementation.

The Tile Framework should remain understandable without mentioning any rendering technology.

Tiles Follow Behaviour

Tiles should never determine behaviour.

Behaviour determines Expressions.

Expressions determine Tiles.

Conceptually.

[text]
Behaviour

↓

Expression

↓

Tile

↓

Component

Responsibility always flows in one direction.

Tiles Preserve Meaning

A Hero Tile should remain a Hero Tile even if:

- rendered differently,
- resized,
- repositioned,
- simplified.

Meaning survives presentation.

This principle enables adaptive interfaces without behavioural fragmentation.

Tiles Are Composable

Tiles intentionally combine into larger experiences.

Example.

[text]
Hero Tile

↓

Timeline Tile

↓

Metadata Tile

↓

Actions Tile

Together they communicate:

Playback

Each Tile owns one responsibility.

The Composition Engine determines how they relate.

Tiles Own Presentation

Expressions communicate:

What exists.

Tiles communicate:

How it should exist physically.

Examples include:

- Material intent
- Typography intent
- Interaction affordance
- Motion behaviour

Rendering frameworks simply implement these characteristics.

Tiles Support Adaptation

A Tile should adapt naturally.

Desktop.



Expanded Hero Tile.

Phone.



Compact Hero Tile.

Voice.



Spoken Hero Tile.

The behavioural meaning remains identical.

Only physical presentation changes.

Tiles Respect Hierarchy

Runtime Hierarchy determines Tile importance.

Hero.



Hero Tile.

Supporting.

↓

Supporting Tile.

Peripheral.

↓

Peripheral Tile.

Tiles should never promote themselves.

Hierarchy always originates from the Composition Engine.

Tiles Support Continuity

Tile identity should survive behavioural evolution.

Example.

Episode 17

↓

Episode 18

The Hero Tile evolves.

It should not disappear and reappear as another unrelated Tile.

Continuity reduces cognitive effort.

Tiles Support Materials

Every Tile inherits Material Intent.

Examples.

Hero Tile.

↓

Hero Material.

Overlay Tile.

↓

Overlay Material.

Supporting Tile.

↓

Surface Material.

Tiles communicate physical identity.

They do not create it.

Tiles Support Typography

Tiles also inherit editorial intent.

Examples.

Hero Tile.



Heading.

Metadata Tile.



Supporting.

Diagnostics Tile.



Caption.

Editorial hierarchy therefore remains consistent throughout the platform.

Tiles Support Motion

Tiles inherit Motion behaviour.

Examples.

Hero Tile.



Hero Motion.

Timeline Tile.



Supporting Motion.

Overlay Tile.



Overlay Motion.

Motion therefore remains behaviourally consistent without components needing independent animation logic.

Plugins

Extensions never define Tiles.

Plugins contribute:

- behaviour,

- information,
- relationships.

The Composition Engine determines Expressions.

The Tile Framework determines Tiles.

This guarantees every extension inherits the same presentation language.

Good Examples

Playback

Expression.

↓

Hero.

↓

Hero Tile.

↓

Platform Component.

The runtime architecture remains intact.

Reading

Expression.

↓

Reading Progress.

↓

Progress Tile.

↓

Presentation.

The same Tile works across every reading experience.

Music

Expression.

↓

Current Track.

↓

Hero Tile.



Television Presentation.

Behaviour remains identical.

Anti-patterns

Component Tiles

Naming Tiles after widgets.

Platform Tiles

Creating Flutter Tiles or React Tiles.

Behavioural Tiles

Allowing Tiles to determine runtime hierarchy.

Plugin Tiles

Extensions introducing independent presentation primitives.

Tile Philosophy Model

```
[mermaid]
flowchart TD
```

```
Behaviour
Behaviour --> Expression
Expression --> Tile
Tile --> Component
Component --> Rendering
Rendering --> User
```

Tiles transform solved understanding into reusable physical presentation.

Relationship To Future Chapters

The following chapters define how this philosophy becomes runtime architecture.

Including:

- Tile Taxonomy
- Expression Mapping
- Tile Lifecycle
- Adaptive Tiles
- Tile Composition

- Tile Interaction
- Runtime Tile Resolution

Every implementation should reinforce the philosophy established here.

Summary

Tiles are the missing bridge between runtime understanding and visible interface.

They ensure that:

- behaviour remains behavioural,
- presentation remains reusable,
- rendering remains replaceable.

Components render Tiles.

Tiles communicate Expressions.

Expressions communicate the user's World.

That layered separation is the defining architectural principle of the Mosaic Tile Framework.

Review Status

Status

Draft

Next File

02-tile-taxonomy.md

Tile Taxonomy

Purpose

The previous chapter established what a Tile is.

This chapter defines the vocabulary of Tiles available throughout Mosaic.

Unlike traditional component libraries, Tile Taxonomy is **not** a catalogue of widgets.

It is a behavioural classification system.

Every Tile exists because it communicates one recurring kind of understanding.

The taxonomy should therefore remain:

- intentionally small,
- behaviourally meaningful,
- presentation independent.

Definition

Within MDS, **Tile Taxonomy** is defined as:

The canonical classification of reusable presentation primitives used to communicate solved runtime Expressions.

Tiles describe:

- purpose,
- behaviour,
- presentation intent.

They intentionally avoid describing:

- implementation,
- layout,
- rendering technology.

Why A Taxonomy Exists

Without a shared taxonomy:

Different contributors naturally invent:

- Hero Card
- Episode Card
- Media Card

- Film Card
- Continue Watching Card

Each solves essentially the same behavioural problem.

The result is duplication.

Instead.

[text]
Hero Tile

One behavioural concept.

Many runtime presentations.

Tile Categories

The current Tile Framework defines the following conceptual Tile families.

[text]
Hero

↓

Content

↓

Timeline

↓

Relationship

↓

Action

↓

Metadata

↓

Collection

↓

Navigation

↓

Overlay

↓

Utility

Every Tile belongs to exactly one primary family.

Hero Tile

Purpose.

Communicate the current Focus.

Examples.

- current film
- current episode
- current book
- current album

Hero Tiles receive:

- Hero Material
- Heading typography
- Hero Motion

There should normally be one Hero Tile within the current Composition.

Content Tile

Purpose.

Represent an individual piece of content.

Examples.

- film
- episode
- chapter
- song
- book

Content Tiles are intentionally neutral.

They become important only through Runtime Hierarchy.

Timeline Tile

Purpose.

Communicate temporal progression.

Examples.

- playback progress
- reading progress

- listening position
- download progress

Timeline Tiles should communicate continuity.

Not merely percentages.

Relationship Tile

Purpose.

Communicate relationships.

Examples.

- sequel
- author
- cast
- franchise
- recommendations

Relationship Tiles strengthen understanding of the user's World.

They should rarely become the behavioural centre.

Action Tile

Purpose.

Communicate behavioural possibility.

Examples.

- continue
- play
- resume
- bookmark
- download

Action Tiles communicate what users can do next.

They should never replace the Hero.

Metadata Tile

Purpose.

Communicate supporting information.

Examples.

- runtime
- codec
- language
- release year
- publisher

Metadata Tiles should remain editorially quiet.

They support understanding.

They rarely lead it.

Collection Tile

Purpose.

Represent organised groups of Expressions.

Examples.

- Continue Watching
- Library
- Recently Added
- Favourites
- Collections

Collection Tiles organise related Content Tiles.

They should not redefine their behavioural importance.

Navigation Tile

Purpose.

Communicate orientation.

Examples.

- Home
- Search
- Library
- Downloads
- Settings

Navigation Tiles remain behaviourally stable.

Users should always know where they are.

Overlay Tile

Purpose.

Support temporary interaction.

Examples.

- search
- playback controls
- command palette
- menus

Overlay Tiles inherit:

- Overlay Material
- Overlay Motion

They remain temporary behavioural participants.

Utility Tile

Purpose.

Communicate supporting system information.

Examples.

- diagnostics
- synchronisation
- storage
- updates

Utility Tiles should remain behaviourally peripheral.

They should never compete with entertainment.

Behavioural Identity

Every Tile possesses one behavioural identity.

Incorrect.

[text]
Hero Timeline Tile

Several responsibilities merged.

Preferred.

[text]
Hero Tile

↓

Timeline Tile

Composition determines relationships.

Tiles remain individually meaningful.

Tile Stability

Tile identities should remain stable for many years.

Example.

Timeline Tile

may later become:

- holographic
- voice
- spatial
- adaptive

Its behavioural identity remains:

Timeline Tile

The taxonomy therefore survives future presentation technologies.

Runtime Hierarchy

Runtime Hierarchy influences Tile importance.

Example.

Content Tile

↓

Hero Tile

A Content Tile may temporarily become the Hero because behaviour changed.

The taxonomy remains unchanged.

Only runtime role evolves.

Material Intent

Each Tile family possesses default Material Intent.

Tile	Material
Hero	Hero Material
Content	Surface Material
Timeline	Acrylic
Relationship	Surface Material

Tile	Material
Action	Acrylic
Metadata	Surface Material
Collection	Canvas / Surface
Navigation	Surface Material
Overlay	Overlay Material
Utility	Surface Material

These defaults may adapt through Runtime Resolution while preserving behavioural identity.

Typography Intent

Each Tile family also carries editorial intent.

Examples.

Hero.



Heading.

Metadata.



Supporting.

Utility.



Caption.

Typography therefore follows Tile behaviour naturally.

Motion Intent

Motion behaviour also derives from Tile family.

Hero.



Hero Motion.

Timeline.



Supporting Motion.

Overlay.



Overlay Motion.

Movement therefore remains behaviourally consistent across every Mosaic client.

Adaptive Behaviour

The same Tile may appear differently across devices.

Desktop.



Expanded Hero Tile.

Phone.



Compact Hero Tile.

Television.



Immersive Hero Tile.

Voice.



Spoken Hero Tile.

The taxonomy remains unchanged.

Plugins

Extensions should never define new Tile families.

Plugins contribute:

- Expressions,
- information,
- relationships.

The Tile Framework maps those Expressions into existing Tile families.

This preserves one coherent presentation vocabulary.

Good Examples

Film

Expression.



Hero.



Hero Tile.



Component.

Reading

Expression.



Bookmarks.



Relationship Tile.



Presentation.

Music

Expression.



Playback Queue.



Collection Tile.



Presentation.

The behavioural identity remains stable.

Anti-patterns

Widget Taxonomy

Naming Tiles after components.

Platform Taxonomy

Creating Flutter Tiles.

Domain Taxonomy

Creating Anime Tiles, Book Tiles or Music Tiles.

Behaviour should remain domain independent.

Plugin Taxonomy

Extensions introducing independent Tile families.

Tile Taxonomy Model

[mermaid]
flowchart TD

```
Expression
Expression --> TileFamily
TileFamily --> Hero
TileFamily --> Content
TileFamily --> Timeline
TileFamily --> Relationship
TileFamily --> Action
TileFamily --> Metadata
TileFamily --> Collection
TileFamily --> Navigation
TileFamily --> Overlay
TileFamily --> Utility
Hero --> Component
Content --> Component
Timeline --> Component
Relationship --> Component
Action --> Component
Metadata --> Component
Collection --> Component
Navigation --> Component
Overlay --> Component
Utility --> Component
```

Expressions determine Tile family.

Components render Tiles.

Relationship To Future Chapters

The next chapter defines **Expression Mapping**.

Tile Taxonomy explains:

Which Tile families exist.

Expression Mapping explains:

How solved Expressions are deterministically mapped into those Tile families at runtime.

Together they establish the runtime presentation vocabulary of Mosaic.

Summary

Tile Taxonomy provides one shared presentation language for the entire Mosaic platform.

It ensures that:

- Expressions remain behavioural,
- Tiles remain reusable,
- Components remain replaceable,
- Extensions remain native.

Every future Mosaic client should therefore communicate one consistent visual language built from the same small vocabulary of behavioural Tiles.

Review Status

Status

Draft

Next File

03-expression-mapping.md

Expression Mapping

Purpose

The Composition Engine produces Expressions.

The Tile Framework produces Tiles.

This chapter defines the deterministic process that connects the two.

Expression Mapping is intentionally one of the strongest architectural boundaries within Mosaic.

Expressions communicate understanding.

Tiles communicate presentation.

Neither system should know implementation details belonging to the other.

Definition

Within MDS, **Expression Mapping** is defined as:

The deterministic process through which solved runtime Expressions are transformed into appropriate Tile identities while preserving behavioural meaning.

Expression Mapping communicates presentation intent.

It does not perform rendering.

Why Mapping Exists

Without Expression Mapping, rendering systems would need to understand runtime concepts directly.

Example.

[text]
Playback

↓

Widgets

This tightly couples behaviour to implementation.

Instead.

[text]
Playback

↓

Expressions

↓

Tiles

↓

Components

↓

Rendering

Every layer owns one responsibility.

One Expression

Every Expression should normally resolve to one primary Tile.

Example.

[text]
Hero

↓

Hero Tile

The Tile may later be implemented differently.

Its behavioural identity remains unchanged.

Behaviour Before Mapping

Mapping should always begin from behavioural meaning.

Incorrect.

[text]
Poster

↓

Poster Card

Correct.

[text]
Hero Expression

↓

Hero Tile

Presentation follows behaviour.

Never the reverse.

Deterministic Mapping

Given identical:

- Runtime World
- Expressions
- Behaviour

- Context

Expression Mapping should always produce identical Tile identities.

This determinism enables:

- caching
- testing
- replay
- cross-platform consistency

Mapping Inputs

Expression Mapping evaluates:

[text]
Expression

↓

Runtime Hierarchy

↓

Behaviour

↓

Context

↓

Device Capability

Rendering technology is intentionally absent.

Tiles remain implementation independent.

Mapping Outputs

The Mapping stage produces:

[text]
Tile Identity

↓

Material Intent

↓

Typography Intent

↓

Motion Intent

↓

Interaction Intent

The Component Library later consumes these outputs.

Hero Mapping

Example.

[text]
Hero Expression

↓

Hero Tile

The Hero Tile inherits:

- Hero Material
- Hero Typography
- Hero Motion

The renderer later decides how this appears physically.

Timeline Mapping

Example.

[text]
Timeline Expression

↓

Timeline Tile

Timeline Tiles inherit:

- Supporting Motion
- Acrylic Material
- Supporting Typography

The behavioural role remains identical across every platform.

Relationship Mapping

Example.

[text]
Relationship Expression

↓

Relationship Tile

Relationship Tiles communicate:

- recommendations
- cast

- author
- franchise

without introducing domain-specific presentation.

Action Mapping

Example.

[text]
Resume Expression

↓

Action Tile

The Tile communicates behavioural opportunity.

Not button implementation.

Buttons remain platform components.

Metadata Mapping

Example.

[text]
Metadata Expression

↓

Metadata Tile

Metadata remains:

- supporting,
- editorial,
- behaviourally quiet.

It should never unexpectedly become primary because of implementation choices.

Runtime Hierarchy

Runtime Hierarchy may influence Tile behaviour.

Example.

[text]
Content Expression

↓

Hero

↓

Hero Tile

The Expression remains:

Content

Its runtime role changes.

Mapping respects that role.

Device Adaptation

Different devices may receive different Tile variants.

Desktop.

↓

Expanded Hero Tile.

Phone.

↓

Compact Hero Tile.

Voice.

↓

Spoken Hero Tile.

These remain behavioural variants of one Tile identity.

The mapping remains conceptually identical.

Material Intent

Expression Mapping also assigns Material Intent.

Example.

[text]
Overlay Expression

↓

Overlay Tile

↓

Overlay Material

Rendering systems remain unaware of behavioural reasoning.

They simply receive Tile metadata.

Typography Intent

Expressions also inherit editorial behaviour.

Example.

[text]

Hero

↓

Hero Tile

↓

Heading

[text]

Metadata

↓

Metadata Tile

↓

Supporting

Typography therefore becomes another consequence of behavioural mapping.

Motion Intent

Motion also follows Tile identity.

Example.

Hero Tile.

↓

Hero Motion.

Timeline Tile.

↓

Supporting Motion.

Overlay Tile.

↓

Overlay Motion.

Movement therefore remains behaviourally consistent without components defining transitions independently.

Interaction Intent

Tiles also communicate interaction.

Examples.

[text]

Hero Tile

↓

Primary Interaction

[text]

Relationship Tile

↓

Exploration

[text]

Metadata Tile

↓

Passive

Interaction therefore emerges naturally from behavioural understanding.

Incremental Mapping

Expression Mapping should update incrementally.

Example.

Playback progress.

↓

Timeline Tile updates.

Hero Tile unchanged.

Only affected mappings should recompute.

This preserves runtime efficiency and behavioural continuity.

Plugins

Extensions contribute:

- Expressions,
- information,
- relationships.

Plugins never perform mapping.

The Tile Framework owns:

- Tile identity,
- Material intent,
- Motion intent,
- Typography intent.

Every extension therefore automatically inherits future presentation improvements.

Good Examples

Playback

Expression.

↓

Timeline.

↓

Timeline Tile.

↓

Platform Component.

Behaviour remains intact.

Reading

Expression.

↓

Bookmarks.

↓

Relationship Tile.

↓

Presentation.

The runtime vocabulary remains consistent.

Music

Expression.

↓

Current Track.

↓

Hero Tile.

↓

Presentation.

Behaviour determines presentation.

Anti-patterns

Widget Mapping

Expressions directly selecting components.

Platform Mapping

Different platforms inventing different Tile identities.

Plugin Mapping

Extensions deciding presentation.

Domain Mapping

Creating Film Tiles, Anime Tiles or Book Tiles.

Behaviour should remain media independent.

Expression Mapping Model

[mermaid]
flowchart TD

```
Expression
Expression --> RuntimeHierarchy
RuntimeHierarchy --> ExpressionMapping
ExpressionMapping --> TileIdentity
TileIdentity --> MaterialIntent
MaterialIntent --> TypographyIntent
TypographyIntent --> MotionIntent
MotionIntent --> Component
```

Expressions determine Tiles.

Tiles determine presentation.

Components remain implementation.

Relationship To Future Chapters

The next chapter defines **Tile Lifecycle**.

Expression Mapping explains:

How Expressions become Tiles.

Tile Lifecycle explains:

How those Tiles evolve over time while preserving behavioural continuity.

Together they establish the runtime life of every presentation primitive within Mosaic.

Summary

Expression Mapping is one of the most important abstraction boundaries within the Mosaic architecture.

It ensures that:

- runtime behaviour remains independent,
- presentation remains reusable,
- rendering remains replaceable.

Expressions describe understanding.

Tiles describe presentation.

Components simply render the result.

Review Status

Status

Draft

Next File

04-tile-lifecycle.md

Tile Lifecycle

Purpose

Expressions continuously evolve as the Runtime World changes.

Tiles must evolve with them.

Unlike conventional UI components, which are frequently created and destroyed in response to interface events, Mosaic Tiles are intended to preserve behavioural continuity.

The Tile Lifecycle defines how Tiles:

- appear,
- evolve,
- persist,
- disappear,

while maintaining one continuous user experience.

The objective is not efficient rendering.

It is uninterrupted understanding.

Definition

Within MDS, **Tile Lifecycle** is defined as:

The behavioural evolution of a Tile from creation through adaptation to retirement while preserving continuity with the Runtime World.

The lifecycle belongs to behaviour.

Not rendering.

Philosophy

Traditional UI frameworks often behave like this.

[text]
Create Widget

↓

Render

↓

Destroy Widget

Mosaic intentionally behaves differently.

[text]
Expression

↓

Tile Appears

↓

Tile Evolves

↓

Tile Adapts

↓

Tile Retires

Users should experience continuous objects rather than disposable interface elements.

Lifecycle Stages

Every Tile progresses through the same conceptual lifecycle.

[text]
Resolved

↓

Materialised

↓

Active

↓

Evolving

↓

Retiring

↓

Released

Each stage communicates one behavioural responsibility.

Stage One

Resolved

The Tile has been selected by Expression Mapping.

At this stage it possesses:

- identity,
- Material intent,
- Typography intent,
- Motion intent,

- Interaction intent.

It does not yet exist visually.

Stage Two

Materialised

The Tile becomes part of the Presentation Model.

Materialised Tiles now possess:

- runtime identity,
- spatial intent,
- hierarchy,
- behavioural role.

Rendering has still not occurred.

Stage Three

Active

The Tile is now visible.

Examples.

Hero Tile.

↓

Displayed.

Timeline Tile.

↓

Displayed.

Metadata Tile.

↓

Displayed.

The Tile participates fully in:

- Motion,
- Materials,
- Interaction,
- Runtime Hierarchy.

Stage Four

Evolving

Most Tiles spend the majority of their lifetime evolving.

Examples.

Playback progresses.

↓

Timeline evolves.

Hero changes.

↓

Hero Tile evolves.

Reading progresses.

↓

Progress Tile evolves.

Evolution should preserve Tile identity whenever practical.

Users should perceive continuity.

Not replacement.

Stage Five

Retiring

A Tile retires when its behavioural purpose ends.

Examples.

Search closes.

↓

Search Tile retires.

Overlay dismissed.

↓

Overlay Tile retires.

The Tile should depart naturally.

It should not simply disappear.

Retirement should preserve behavioural understanding.

Stage Six

Released

The Tile is no longer behaviourally relevant.

Runtime resources may now be reclaimed.

Importantly...

Users should already understand why the Tile disappeared before this stage occurs.

Release is an implementation concern.

Retirement is a behavioural concern.

Behaviour Owns Lifecycle

Only behaviour changes Tile lifecycle.

Incorrect.

[text]
Scroll

↓

Destroy Tile

Correct.

[text]
Behaviour Changes

↓

Tile Retires

Rendering optimisations should never redefine behavioural lifetime.

Identity Preservation

Whenever practical...

A Tile should evolve rather than be replaced.

Preferred.

[text]
Episode 17

↓

Episode 18

↓

Hero Tile Evolves

Avoid.

[text]

Hero Tile Destroyed

↓

New Hero Tile Created

Identity continuity significantly reduces cognitive effort.

Runtime Hierarchy

Hierarchy changes may alter a Tile's role.

Example.

Content Tile

↓

Hero Tile

The Tile remains behaviourally related.

Only its runtime importance evolves.

This distinction strengthens continuity throughout the interface.

Material Continuity

Tiles should preserve Material identity throughout evolution.

Hero Tile.

↓

Hero Material.

↓

Still Hero Material.

A Tile should never unexpectedly become an unrelated material because implementation changed.

Typography Continuity

Editorial roles should remain stable.

Examples.

Heading.

↓

Heading.

Supporting.

↓

Supporting.

The wording may change.

The editorial language should remain recognisable.

Motion Continuity

Tile lifecycle transitions should follow the Motion System.

Examples.

Resolved.

↓

Materialised.

↓

Active.

↓

Evolving.

↓

Retiring.

↓

Released.

Movement communicates behavioural evolution.

It should never obscure it.

Incremental Evolution

Small behavioural updates should produce small lifecycle changes.

Example.

Playback progresses.

↓

Timeline Tile evolves.

Everything else remains active.

Incremental evolution preserves runtime stability while reducing unnecessary work.

Reuse

Future implementations may reuse Tile instances.

Conceptually.

[text]

Released Tile



Compatible Expression



Reuse

Reuse is an implementation optimisation.

The behavioural lifecycle remains unchanged.

Runtime Ownership

The Tile Framework owns lifecycle.

Components merely render current lifecycle state.

Rendering frameworks should never independently determine:

- creation,
- retirement,
- evolution.

Behaviour remains authoritative.

Plugins

Extensions contribute:

- Expressions,
- behaviour,
- information.

Plugins never manage Tile lifecycles.

Every extension therefore inherits one consistent behavioural model automatically.

Good Examples

Playback

Timeline Tile.



Progress updates.



Tile evolves.



Playback continues.

No unnecessary replacement occurs.

Reading

Bookmark added.

↓

Relationship Tile evolves.

↓

Reader continues uninterrupted.

Search

Search Tile appears.

↓

Interaction.

↓

Search Tile retires.

↓

Previous Composition continues naturally.

Anti-patterns

Widget Lifecycle

Component creation determining behavioural lifetime.

Instant Replacement

Destroying Tiles rather than evolving them.

Platform Lifecycle

Different clients managing behavioural lifetime differently.

Plugin Lifecycle

Extensions owning Tile creation and destruction.

Tile Lifecycle Model

[mermaid]
flowchart TD

Expression
Expression --> Resolved
Resolved --> Materialised
Materialised --> Active
Active --> Evolving
Evolving --> Retiring
Retiring --> Released

Tiles evolve because behaviour evolves.

Rendering simply follows.

Relationship To Future Chapters

The next chapter defines **Adaptive Tiles**.

Tile Lifecycle explains:

How Tiles evolve over time.

Adaptive Tiles explain:

How the same Tile adapts across different environments while preserving its behavioural identity.

Together they ensure Tiles remain both continuous and adaptive.

Summary

The Tile Lifecycle ensures presentation behaves like the rest of Mosaic.

Continuous.

Behavioural.

Predictable.

Tiles should not feel disposable.

They should feel like persistent participants within the user's World, evolving naturally as behaviour changes and quietly departing only when their purpose has truly ended.

Review Status

Status

Draft

Next File

05-adaptive-tiles.md

Adaptive Tiles

Purpose

Every Tile communicates one behavioural idea.

That idea must remain recognisable regardless of:

- device,
- screen size,
- viewing distance,
- interaction method,
- accessibility preferences.

Adaptive Tiles define how one Tile identity evolves across different presentation environments without changing its behavioural meaning.

The Tile adapts.

The understanding remains constant.

Definition

Within MDS, **Adaptive Tiles** are defined as:

Tiles capable of altering their physical presentation while preserving behavioural identity, runtime hierarchy and conceptual responsibility.

Adaptive behaviour changes presentation.

It never changes purpose.

Philosophy

Traditional responsive systems frequently create:

- desktop cards,
- mobile cards,
- TV cards.

Each gradually evolves into an independent implementation.

Mosaic intentionally avoids this.

Instead.

[text]
Hero Tile

↓

Adaptive Hero Tile

↓

Platform Presentation

One Tile.

Many physical expressions.

Behaviour Before Adaptation

Adaptive Tiles always begin with behaviour.

Incorrect.

[text]
Phone

↓

Compact Card

Correct.

[text]
Hero Tile

↓

Adaptive Projection

↓

Phone Presentation

Devices influence presentation.

They never redefine behaviour.

One Tile Identity

Every Adaptive Tile preserves one stable identity.

Examples.

[text]
Hero Tile

↓

Desktop

[text]
Hero Tile

↓

Phone

[text]
Hero Tile

↓

Television

Every presentation remains:

Hero Tile

The behavioural identity never changes.

Adaptive Inputs

Adaptive behaviour evaluates:

[text]

Tile Identity

↓

Runtime Hierarchy

↓

Device Class

↓

Viewing Distance

↓

Accessibility

↓

Capabilities

Behaviour has already been solved.

Adaptive Tiles communicate that behaviour appropriately.

Adaptive Outputs

Adaptive Tiles produce:

[text]

Presentation Variant

↓

Material Behaviour

↓

Typography Behaviour

↓

Interaction Behaviour

↓

Layout Constraints

These outputs remain independent from rendering technology.

Hero Adaptation

Desktop.



Expanded Hero.

Phone.



Compact Hero.

Television.



Immersive Hero.

Voice.



Spoken Hero.

The Hero remains behaviourally identical.

Only presentation differs.

Timeline Adaptation

Timeline Tiles should adapt according to available space.

Examples.

Desktop.



Expanded progress.

Phone.



Condensed progress.

Television.



Distance-optimised progress.

Voice.



Spoken progress.

Users should always recognise:

Timeline

regardless of implementation.

Metadata Adaptation

Metadata should progressively collapse.

Preferred.

Desktop.

↓

Complete metadata.

Phone.

↓

Primary metadata.

↓

Expandable detail.

Understanding remains available.

Presentation simply becomes quieter.

Relationship Adaptation

Relationship Tiles may adapt by:

- grouping,
- collapsing,
- summarising,
- expanding.

Relationships should never disappear solely because presentation space decreased.

Behaviour always possesses higher priority than convenience.

Collection Adaptation

Collections should adapt naturally.

Desktop.

↓

Large collection.

Phone.

↓

Progressive collection.

Television.



Immersive collection.

The Collection Tile remains behaviourally unchanged.

Material Adaptation

Adaptive Tiles inherit Material behaviour.

Examples.

Hero Tile.



Hero Material.

Compact Hero.



Simplified Hero Material.

Low Power Device.



Reduced Material Fidelity.

Material identity remains recognisable.

Only rendering complexity changes.

Typography Adaptation

Editorial hierarchy remains stable.

Heading.



Heading.

Supporting.



Supporting.

Caption.



Caption.

Adaptive Tiles may alter:

- measure,

- spacing,
- line length.

They should never alter editorial meaning.

Motion Adaptation

Motion should adapt naturally.

Examples.

Desktop.

↓

Full Material Motion.

Phone.

↓

Reduced travel.

Television.

↓

Greater perceived distance.

Reduced Motion.

↓

Minimal movement.

Behavioural sequencing remains identical.

Interaction Adaptation

Interaction methods naturally differ.

Examples.

Phone.

↓

Touch.

Desktop.

↓

Pointer.

Television.

↓

Remote.

Voice.



Conversation.

Adaptive Tiles expose different interaction affordances.

The underlying behaviour remains identical.

Accessibility

Accessibility may adapt Tile presentation.

Examples.

Large Text.



Greater spacing.

Reduced Motion.



Simplified transitions.

High Contrast.



Clearer Materials.

The Tile should remain behaviourally recognisable under every accessibility profile.

Runtime Adaptation

Adaptive behaviour should occur continuously.

Examples.

Window resized.



Tile evolves.

Orientation changes.



Tile adapts.

Accessibility enabled.



Tile refines.

The Tile should never appear to restart its lifecycle because presentation changed.

Device Independence

Adaptive behaviour should remain independent from specific platforms.

Future devices should require only:

Capability Profile

↓

Adaptive Projection

The Tile Framework should remain prepared for technologies that do not yet exist.

Plugins

Extensions never define adaptive behaviour.

Plugins contribute:

- Expressions,
- behaviour,
- information.

Adaptive Tiles remain entirely platform owned.

Every extension therefore automatically supports future devices.

Good Examples

Hero

Desktop.

↓

Expanded Hero.

Phone.

↓

Compact Hero.

Behaviour remains identical.

Reading

Metadata collapses naturally.

↓

Reader expands when required.

↓

Understanding preserved.

Television

Collection Tile expands.

↓

Greater spacing.

↓

Long-distance readability.

↓

Same behavioural identity.

Anti-patterns

Device Tiles

Creating independent Mobile Hero Tiles.

Layout Behaviour

Changing behavioural meaning because layout changed.

Platform Identity

Different clients inventing different Tile vocabularies.

Accessibility Tiles

Creating separate Tile identities for accessibility.

Adaptive Tile Model

[mermaid]
flowchart TD

```
TileIdentity
TileIdentity --> AdaptiveProjection
AdaptiveProjection --> DeviceProfile
DeviceProfile --> Desktop
DeviceProfile --> Phone
DeviceProfile --> Television
DeviceProfile --> Voice
Desktop --> Presentation
Phone --> Presentation
Television --> Presentation
Voice --> Presentation
```

One Tile identity.

Many adaptive presentations.

Relationship To Future Chapters

The next chapter defines **Tile Composition**.

Adaptive Tiles explain:

*How one **Tile** adapts across environments.*

Tile Composition explains:

*How multiple **Tiles** combine into coherent runtime presentation while preserving behavioural understanding.*

Together they establish the presentation architecture of Mosaic.

Summary

Adaptive Tiles ensure that Mosaic remains:

- behaviourally consistent,
- presentation flexible,
- future proof.

Users should never think:

"This is the mobile version."

They should simply feel that the same Companion naturally adapted to the device they chose to use.

That behavioural continuity is the defining objective of Adaptive Tiles.

Review Status

Status

Draft

Next File

06-tile-composition.md

Tile Composition

Purpose

Individual Tiles communicate individual Expressions.

Real user experiences, however, are rarely expressed through one Tile alone.

Tile Composition defines how multiple Tiles combine into coherent behavioural experiences while preserving:

- hierarchy,
- continuity,
- understanding,
- adaptability.

Unlike traditional layout systems, Tile Composition is not concerned with positioning widgets.

It is concerned with composing understanding.

Definition

Within MDS, **Tile Composition** is defined as:

The behavioural organisation of multiple Tiles into coherent presentation structures that preserve the solved understanding produced by the Composition Engine.

Tile Composition is behavioural.

Layout is implementation.

Philosophy

Traditional interfaces frequently compose components.

Examples.

[text]
Card

↓

Button

↓

List

↓

Grid

Mosaic intentionally composes behavioural concepts.

[text]
Hero Tile

↓

Timeline Tile

↓

Metadata Tile

↓

Relationship Tile

Presentation emerges naturally from understanding.

Composition Before Layout

Tile Composition should never begin by asking:

Where should this Tile go?

Instead ask:

Which Tiles belong together behaviourally?

Grouping always precedes geometry.

Behaviour Creates Composition

Every Tile Composition originates from runtime behaviour.

Example.

Playback

↓

Hero

↓

Timeline

↓

Actions

↓

Metadata

These Tiles form one behavioural experience.

The runtime determines placement afterwards.

Composition Groups

Tile Composition naturally forms behavioural groups.

Examples include:

[text]

Hero Group

Playback Group

Reading Group

Discovery Group

Relationship Group

Utility Group

Each group communicates one behavioural purpose.

Groups should remain conceptually independent.

Hero Group

The Hero Group normally contains:

[text]

Hero Tile

↓

Timeline Tile

↓

Primary Action Tile

↓

Metadata Tile

The Hero Group represents the behavioural centre of the current World.

Nothing outside the group should compete for primary attention.

Reading Group

A Reading Group may contain:

[text]

Hero Tile

↓

Progress Tile

↓

Bookmarks

↓

Chapter Navigation

Reading behaviour determines composition.

Not media type.

Playback Group

Playback naturally groups:

[text]
Hero

↓

Timeline

↓

Playback Actions

↓

Audio

↓

Subtitles

The group should communicate one coherent behavioural task.

Relationship Group

Relationship Tiles naturally compose together.

Examples.

[text]
Cast

↓

Studio

↓

Recommendations

↓

Related Works

Relationships should strengthen discovery.

Not interrupt the current Focus.

Utility Group

Utility information should compose separately.

Examples.

[text]
Diagnostics

↓

Downloads

↓

Sync Status

Utility groups should remain behaviourally peripheral.

Entertainment should remain central.

Hierarchical Composition

Tile Composition should respect Runtime Hierarchy.

Example.

[text]
Hero Group

↓

Supporting Groups

↓

Peripheral Groups

Groups should never compete equally.

Behavioural importance should remain immediately recognisable.

Nested Composition

Groups may contain sub-groups.

Example.

[text]
Hero Group

■■■ Hero

■■■ Playback

■■■ Metadata

Nested groups improve organisation without changing behavioural meaning.

Composition Stability

Tile groups should remain stable whenever possible.

Example.

Playback progress updates.

↓

Timeline Group evolves.

Hero Group remains intact.

Stable composition significantly improves user orientation.

Material Cohesion

Tiles within the same group should feel materially related.

Example.

Hero Group.

↓

Hero Material.

↓

Supporting Acrylic.

↓

Shared Atmosphere.

Groups should feel physically connected.

Not visually assembled.

Typography Cohesion

Editorial hierarchy should remain consistent within groups.

Example.

[text]
Hero

↓

Heading

Timeline

↓

Supporting

Metadata

↓

Caption

Typography should reinforce group structure naturally.

Motion Cohesion

Tile groups should move together behaviourally.

Example.

Hero changes.

↓

Hero Group evolves.

↓

Relationship Group responds.

↓

Environment settles.

Movement should reinforce conceptual grouping.

Not merely physical proximity.

Adaptive Composition

Adaptive Layout should preserve Tile groups.

Desktop.

↓

Expanded Hero Group.

Phone.

↓

Compact Hero Group.

Television.

↓

Immersive Hero Group.

The group remains behaviourally identical.

Only spatial expression changes.

Runtime Composition

Tile Composition should evolve incrementally.

Example.

Bookmark added.

↓

Reading Group updates.

Playback Group unchanged.

Behavioural locality preserves continuity.

Composition Integrity

Tiles should never migrate between unrelated groups purely because layout changed.

Behaviour owns grouping.

Presentation communicates it.

Plugins

Extensions contribute Expressions.

The Tile Framework determines:

- Tile identity,
- Tile groups,
- behavioural composition.

Plugins should never construct Tile groups independently.

Every extension therefore inherits one coherent presentation language.

Good Examples

Playback

Hero Group.

↓

Playback Group.

↓

Relationship Group.

↓

Utility Group.

Behaviour remains immediately understandable.

Reading

Hero Group.

↓

Reading Group.

↓

Relationship Group.

↓

Bookmarks.

Readers naturally understand where to look.

Music

Hero Group.

↓

Queue Group.

↓

Recommendations.

↓

Metadata.

Listening behaviour determines composition.

Anti-patterns

Layout Groups

Grouping Tiles purely because they fit inside the same container.

Component Groups

Widgets determining behavioural organisation.

Platform Groups

Different clients inventing different behavioural groupings.

Plugin Groups

Extensions constructing independent presentation structures.

Tile Composition Model

```
[mermaid]
flowchart TD
```

```
Expressions
Expressions --> Tiles
Tiles --> BehaviourGroups
BehaviourGroups --> AdaptiveLayout
AdaptiveLayout --> Presentation
Presentation --> Rendering
```

Behaviour creates groups.

Groups create presentation.

Rendering simply expresses the result.

Relationship To Future Chapters

The next chapter defines **Tile Interaction**.

Tile Composition explains:

How Tiles belong together.

Tile Interaction explains:

How users interact with those Tiles while preserving the behavioural language established by the Composition Engine.

Together they complete the conceptual architecture of the Tile Framework.

Summary

Tile Composition ensures that presentation remains behaviourally meaningful.

Tiles should never feel like unrelated interface elements arranged on a screen.

Instead they should feel like members of one coherent conversation about the user's current World.

That behavioural cohesion is one of the defining characteristics of the Mosaic Tile Framework.

Review Status

Status

Draft

Next File

07-tile-interaction.md

Tile Interaction

Purpose

Tiles communicate behavioural understanding.

Interaction allows users to participate in that behaviour.

Unlike traditional component systems, where interaction is owned by widgets, Mosaic intentionally assigns interaction to Tiles.

Components merely render interaction.

Tiles communicate behavioural intent.

This distinction preserves one interaction language across every platform.

Definition

Within MDS, **Tile Interaction** is defined as:

The behavioural contract through which users interact with a Tile while preserving the integrity of the Runtime World and Composition Engine.

Interaction belongs to behaviour.

Not presentation.

Philosophy

Traditional interfaces frequently model interaction like this.

[text]
Button

↓

Click

↓

Callback

Mosaic intentionally models interaction differently.

[text]
Tile

↓

Behavioural Intent

↓

Runtime Behaviour

↓

Composition Evolves

Users interact with understanding.

Not components.

Interaction Before Components

Components should never own behaviour.

Incorrect.

[text]
Button

↓

Play

Correct.

[text]
Action Tile

↓

Resume Playback

The component merely renders the Action Tile.

The Tile owns behavioural meaning.

Interaction Categories

The Tile Framework recognises several conceptual interaction categories.

[text]
Primary

↓

Secondary

↓

Exploration

↓

Navigation

↓

Contextual

↓

Passive

Every Tile exposes one primary interaction category.

Primary Interaction

Purpose.

Advance the user's current behavioural goal.

Examples.

- Resume playback
- Continue reading
- Play album
- Open Hero

Primary interactions should always reinforce the current Focus.

They should never compete with it.

Secondary Interaction

Purpose.

Support the current behaviour.

Examples.

- Bookmark
- Favourite
- Queue
- Download

Secondary interactions remain immediately available without competing with the primary behavioural objective.

Exploration Interaction

Purpose.

Expand the user's World.

Examples.

- Cast member
- Related work
- Author
- Franchise

Exploration interactions encourage discovery.

They should never interrupt the current behavioural flow.

Navigation Interaction

Purpose.

Move between behavioural contexts.

Examples.

- Home
- Search
- Library
- Downloads

Navigation should preserve continuity.

Users should always feel they remain within one World.

Contextual Interaction

Purpose.

Reveal additional behavioural capability.

Examples.

- Context menu
- More actions
- Diagnostics
- Advanced controls

Contextual interactions should remain behaviourally subordinate.

They should never obscure primary understanding.

Passive Interaction

Some Tiles intentionally expose no direct interaction.

Examples.

- informational metadata
- decorative atmosphere
- environmental elements

Passive Tiles contribute understanding without requiring user action.

Not every Tile should invite interaction.

Behavioural Intent

Every interaction communicates behavioural intent.

Example.

[text]
Resume Tile

↓

Resume Playback

Not.

[text]
Pressed Button

The Runtime World consumes behavioural intent.

Implementation events remain platform concerns.

Interaction Resolution

Tile Interaction resolves into behavioural events.

Conceptually.

[text]
Tile

↓

Interaction Intent

↓

Behaviour

↓

Runtime World

↓

Composition

Tiles never mutate the Runtime World directly.

They communicate behavioural intent.

Hero Interaction

The Hero Tile normally exposes one clear primary interaction.

Examples.

- Play
- Resume
- Continue

- Open

Multiple competing primary interactions weaken behavioural clarity.

The Hero should answer one question.

"What should I do next?"

Relationship Interaction

Relationship Tiles encourage exploration.

Examples.

Cast.



Actor details.

Series.



Season.

Author.



Other works.

Exploration should always preserve orientation.

Users should never feel lost after following relationships.

Timeline Interaction

Timeline Tiles expose temporal interaction.

Examples.

- seek
- chapter selection
- reading progress
- queue position

Timeline interactions should preserve continuity.

They modify the current World.

They do not replace it.

Overlay Interaction

Overlay Tiles temporarily become the interaction centre.

Examples.

- Search
- Playback controls
- Settings
- Menus

When the Overlay retires:

The user's previous behavioural context should immediately return.

Interaction should never permanently fragment the Runtime World.

Material Behaviour

Interactions should respect Material identity.

Hero Tile.



Hero Material response.

Overlay Tile.



Overlay Material response.

Metadata Tile.



Minimal material response.

Physical behaviour reinforces interaction confidence.

Typography Behaviour

Editorial hierarchy should remain stable throughout interaction.

Interactions may alter:

- emphasis,
- focus,
- selection.

They should never unexpectedly change editorial roles.

Understanding always remains stable.

Motion Behaviour

Interactions initiate behavioural Motion.

Example.

Interaction.



Behaviour changes.



Motion communicates behaviour.

Movement never exists independently from interaction.

Accessibility

Every interaction should remain accessible through:

- keyboard
- remote
- touch
- screen reader
- voice

Interaction categories remain identical.

Only implementation changes.

Multi-Device Interaction

The same Tile may expose different physical interaction methods.

Desktop.



Pointer.

Phone.



Touch.

Television.



Remote.

Voice.



Conversation.

The behavioural outcome remains identical.

Runtime Ownership

The Runtime World owns behaviour.

Tiles own interaction intent.

Components own implementation.

Responsibilities remain intentionally separated.

Plugins

Extensions contribute:

- behavioural capabilities
- interaction metadata

Plugins never implement interaction directly.

The Tile Framework determines:

- interaction category
- runtime behaviour
- presentation

Every extension therefore inherits the same interaction language.

Good Examples

Playback

Hero Tile.

↓

Resume.

↓

Runtime Behaviour.

↓

Playback.

↓

Composition evolves.

The interaction feels inevitable.

Reading

Bookmark Tile.

↓

Bookmark Behaviour.

↓

Reading continues.

↓

World preserved.

Discovery

Relationship Tile.

↓

Explore Author.

↓

New behavioural context.

↓

Continuity maintained.

Anti-patterns

Component Behaviour

Widgets directly mutating application state.

Multiple Primary Actions

Several equally important actions competing for attention.

Platform Interaction

Different clients inventing different behavioural outcomes.

Plugin Behaviour

Extensions bypassing the Runtime World.

Tile Interaction Model

[mermaid]
flowchart TD

```
Tile
Tile --> InteractionIntent
InteractionIntent --> Behaviour
Behaviour --> RuntimeWorld
RuntimeWorld --> Composition
Composition --> Presentation
Presentation --> Rendering
```

Interaction begins with Tiles.

Behaviour remains the ultimate authority.

Relationship To Future Chapters

The next chapter defines **Runtime Tile Resolution**.

Tile Interaction explains:

How users communicate with Tiles.

Runtime Tile Resolution explains:

How Tiles become fully resolved runtime presentation objects before rendering.

Together they complete the behavioural architecture of runtime presentation.

Summary

Tile Interaction ensures that every user interaction remains:

- behavioural,
- predictable,
- continuous,
- platform independent.

Users should never feel they are clicking widgets.

They should feel they are naturally interacting with their current World.

That distinction is one of the defining architectural principles of the Mosaic Tile Framework.

Review Status

Status

Draft

Next File

08-runtime-tile-resolution.md

Runtime Tile Resolution

Purpose

The Tile Framework defines:

- Tile identities,
- Tile behaviour,
- Tile interaction,
- Tile composition.

This chapter defines how those concepts become concrete runtime presentation objects.

Runtime Tile Resolution transforms behavioural Tiles into fully resolved presentation primitives ready for implementation by the Component Library.

Components should never decide:

- Material
- Typography
- Motion
- Interaction
- Layout

They receive already resolved Tiles.

Definition

Within MDS, **Runtime Tile Resolution** is defined as:

The deterministic process through which behavioural Tile identities become fully resolved runtime presentation objects while preserving behavioural meaning and platform independence.

Runtime Tile Resolution resolves presentation.

It never changes behaviour.

Why Resolution Exists

Without Runtime Tile Resolution, every component would need to understand:

- Runtime Hierarchy
- Material Intent
- Typography Intent
- Motion Intent
- Adaptive behaviour

- Accessibility

Instead.

[text]
Tile

↓

Runtime Tile Resolver

↓

Resolved Tile

↓

Component

Components remain extremely simple.

Resolution Pipeline

Every Tile follows the same conceptual pipeline.

[text]
Tile Identity

↓

Runtime Hierarchy

↓

Material Intent

↓

Typography Intent

↓

Motion Intent

↓

Accessibility

↓

Adaptive Behaviour

↓

Resolved Tile

Each stage contributes one responsibility.

Resolution Inputs

Runtime Tile Resolution evaluates:

[text]
Tile Identity

↓

Expression

↓

Runtime Hierarchy

↓

Current Context

↓

Device Profile

↓

Accessibility

↓

Capabilities

Rendering technology remains intentionally absent.

Resolution Outputs

Resolved Tiles contain:

[text]
Material

↓

Typography

↓

Motion

↓

Interaction

↓

Adaptive Variant

↓

Presentation Metadata

These objects become the direct input to the Component Library.

Tile Identity Is Stable

One of the strongest guarantees within Mosaic is:

[text]
Hero Tile

↓

Always Hero Tile

Runtime Resolution may alter:

- material richness,
- layout,
- interaction affordances,
- typography.

It never alters Tile identity.

Material Resolution

Material Intent resolves into runtime Materials.

Example.

[text]
Hero Tile

↓

Hero Material

↓

Resolved Hero Material

Components should never determine Material behaviour independently.

Typography Resolution

Typography Intent resolves similarly.

Example.

[text]
Metadata Tile

↓

Supporting Typography

↓

Resolved Typography

Editorial hierarchy therefore remains entirely runtime driven.

Motion Resolution

Motion Intent resolves into runtime behaviour.

Example.

[text]
Timeline Tile

↓

Supporting Motion

↓

Resolved Motion Profile

Components consume motion.

They never construct it.

Interaction Resolution

Interaction Intent also resolves.

Examples.

[text]
Primary

↓

Touch

↓

Mouse

↓

Remote

↓

Voice

Interaction methods adapt.

Behavioural intent remains unchanged.

Adaptive Resolution

Runtime Tile Resolution also determines adaptive variants.

Example.

Desktop.

↓

Expanded Hero Tile.

Phone.

↓

Compact Hero Tile.

Voice.

↓

Conversational Hero Tile.

The Tile remains behaviourally identical.

Accessibility Resolution

Accessibility refines resolved Tiles.

Examples.

Large Text.

↓

Typography.

Reduced Motion.

↓

Motion.

High Contrast.

↓

Materials.

Behaviour remains identical.

Only presentation adapts.

Runtime Profiles

Future implementations may internally generate Tile Profiles.

Conceptually.

[text]
Hero Tile

↓

Tile Profile

↓

Material

↓

Typography

↓

Motion

↓

Interaction

↓

Resolved Tile

Components consume Tile Profiles.

They remain unaware of runtime solving.

Runtime Caching

Resolved Tiles should be aggressively cacheable.

Typical invalidation events include:

- behaviour changes,
- hierarchy changes,
- accessibility changes,
- device changes.

Ordinary rendering should reuse existing Tile Profiles whenever practical.

Incremental Resolution

Preferred.

[text]
Timeline Tile

↓

Resolved

↓

Updated

Avoid.

[text]
Every Tile

↓

Resolved Again

Incremental resolution preserves runtime performance and behavioural continuity.

Platform Independence

Runtime Tile Resolution should remain completely platform independent.

Flutter.

↓

Consumes Resolved Tile.

Web.

↓

Consumes Resolved Tile.

SwiftUI.



Consumes Resolved Tile.

Compose.



Consumes Resolved Tile.

Presentation differs.

Resolved behavioural intent remains identical.

Deterministic Resolution

Given identical:

- Tile Identity,
- Runtime World,
- Hierarchy,
- Accessibility,

Runtime Tile Resolution should always produce identical resolved Tiles.

Determinism improves:

- caching,
- replay,
- testing,
- cross-platform consistency.

Plugins

Extensions contribute:

- Expressions,
- behaviour,
- relationships.

Plugins never resolve Tiles.

The Tile Framework owns:

- Tile Resolution,
- adaptive behaviour,
- runtime presentation.

Every extension therefore automatically inherits future Tile improvements.

Good Examples

Playback

Timeline Tile.



Runtime Resolution.



Resolved Timeline.



Component.

Behaviour remains preserved.

Reading

Relationship Tile.



Adaptive Resolution.



Phone Presentation.



Reader continues naturally.

Television

Hero Tile.



Immersive Variant.



Hero Material.



Presentation.

One behavioural identity.

Anti-patterns

Component Resolution

Components selecting Materials independently.

Platform Resolution

Each client inventing Tile behaviour.

Widget Resolution

Rendering technology influencing behavioural presentation.

Plugin Resolution

Extensions bypassing the Tile Framework.

Runtime Tile Resolution Model

[mermaid]
flowchart TD

```
TileIdentity
TileIdentity --> RuntimeTileResolver
RuntimeTileResolver --> Material
RuntimeTileResolver
RuntimeTileResolver --> Typography
RuntimeTileResolver
RuntimeTileResolver --> Motion
RuntimeTileResolver
RuntimeTileResolver --> Interaction
Material --> ResolvedTile
Typography --> ResolvedTile
Motion --> ResolvedTile
Interaction --> ResolvedTile
ResolvedTile
ResolvedTile --> Component
```

The Runtime Tile Resolver transforms behavioural presentation into implementation-ready runtime objects.

Relationship To Future Chapters

The next chapter defines **Extension Tiles**.

Runtime Tile Resolution explains:

How Tiles become runtime presentation.

Extension Tiles explain:

How third-party runtime contributions participate in the Tile Framework while remaining indistinguishable from native Mosaic presentation.

Together they complete the runtime architecture of the Tile Framework.

Summary

Runtime Tile Resolution ensures every behavioural Tile becomes one fully resolved presentation object before rendering begins.

Components therefore remain implementation details.

Behaviour remains the architectural authority.

Tiles remain the presentation language of Mosaic.

That separation allows the platform to evolve indefinitely while preserving one coherent runtime experience.

Review Status

Status

Draft

Next File

09-extension-tiles.md

Extension Tiles

Purpose

One of Mosaic's defining architectural goals is allowing extensions to feel indistinguishable from native functionality.

Users should never be able to identify whether a Tile originated from:

- Mosaic Core,
- an official extension,
- a community extension,
- a third-party integration.

This chapter defines how extensions participate in the Tile Framework without fragmenting the behavioural language of the platform.

Extensions contribute understanding.

The Tile Framework determines presentation.

Definition

Within MDS, **Extension Tiles** are defined as:

Runtime Tiles produced from extension-provided Expressions using the same behavioural resolution pipeline as native Mosaic functionality.

Extension Tiles are not custom Tile types.

They are standard Mosaic Tiles whose behavioural source happens to originate from an extension.

Philosophy

Traditional plugin systems frequently allow extensions to create arbitrary interfaces.

The result is:

- inconsistent layouts,
- inconsistent motion,
- inconsistent typography,
- inconsistent interaction.

Mosaic intentionally rejects this model.

Instead:

[text]
Extension

↓

Expression

↓

Tile Framework

↓

Standard Tile

↓

Presentation

Extensions become behaviourally native.

Extensions Never Create Tiles

This principle is fundamental.

Extensions provide:

- information,
- relationships,
- behaviours,
- capabilities.

They never provide:

- Hero Tiles,
- custom Cards,
- layouts,
- widgets.

The Tile Framework remains solely responsible for presentation.

Behaviour Before Presentation

Extensions communicate behavioural meaning.

Example.

[text]
Real-Debrid Stream

↓

Playback Expression

Not:

[text]
Streaming Widget

The runtime determines how playback should appear.

The extension merely enriches the Runtime World.

Expression Ownership

Extension Expressions participate identically to native Expressions.

Example.

[text]
Extension

↓

Timeline Expression

↓

Timeline Tile

↓

Platform Component

The Composition Engine never distinguishes between native and extension Expressions during presentation.

Only behaviour matters.

Tile Resolution

Extension Expressions pass through the complete runtime pipeline.

[text]
Extension

↓

Runtime World

↓

Composition Solver

↓

Expression Resolution

↓

Tile Resolution

↓

Presentation

No shortcuts should exist.

This guarantees behavioural consistency.

Runtime Hierarchy

Extensions never assign hierarchy.

Incorrect.

[text]
Extension

↓

Hero

Correct.

[text]
Extension

↓

Expression

↓

Composition Solver

↓

Hero

Only the Composition Engine determines runtime importance.

This preserves behavioural integrity.

Material Behaviour

Extension Tiles inherit standard Material behaviour.

Examples.

Hero.

↓

Hero Material.

Relationship.

↓

Surface Material.

Overlay.

↓

Overlay Material.

Extensions should never specify Material behaviour directly.

Typography Behaviour

Editorial hierarchy also remains platform owned.

Examples.

Extension Metadata.

↓

Supporting.

Extension Hero.



Heading.

Extension Diagnostics.



Caption.

The editorial language remains consistent regardless of content source.

Motion Behaviour

Extension Tiles inherit standard Motion.

Examples.

Playback.



Hero Motion.

Relationship.



Supporting Motion.

Overlay.



Overlay Motion.

Extensions never define transitions.

Movement remains behaviourally consistent.

Adaptive Behaviour

Extension Tiles adapt exactly like native Tiles.

Desktop.



Expanded Tile.

Phone.



Compact Tile.

Television.



Immersive Tile.

Voice.



Conversational Tile.

Extensions automatically support future presentation environments.

Interaction Behaviour

Extensions communicate interaction intent.

Examples.

[text]
Play

Bookmark

Install

Open

The Tile Framework resolves:

- touch,
- pointer,
- remote,
- keyboard,
- voice.

Interaction therefore remains platform independent.

Extension Identity

Users should perceive:

Mosaic

Not:

Extension UI

Visual identity belongs to the platform.

Behaviour belongs to the Runtime World.

Extensions should therefore disappear into the experience.

Capability Projection

Extensions may expose capabilities.

Examples.

[text]
Offline Download

Transcoding

Metadata

Streaming

Recommendations

Capabilities become behavioural Expressions.

The Composition Engine determines whether and how they appear.

Capability does not imply presentation.

Runtime Updates

Extension updates should follow identical runtime behaviour.

Example.

Extension updates metadata.

↓

Runtime World updates.

↓

Composition evolves.

↓

Tiles update.

The extension should never directly manipulate presentation.

Accessibility

Extension Tiles automatically inherit:

- accessibility,
- adaptive layout,
- motion preferences,
- typography,
- materials.

Extensions should never implement accessibility independently.

The platform guarantees consistency.

Multi-Device Behaviour

Extension Tiles should remain behaviourally identical across:

- desktop,
- mobile,
- television,
- voice,
- future platforms.

Presentation adapts.

Behaviour remains stable.

Extensions therefore automatically support future clients without modification.

Security Boundary

The Tile Framework intentionally provides an architectural security boundary.

Extensions should never directly influence:

- rendering,
- runtime hierarchy,
- Materials,
- Typography,
- Motion,
- Adaptive Layout.

This significantly reduces the risk of inconsistent or malicious presentation behaviour.

Runtime Ownership

The Runtime World owns:

- hierarchy,
- composition,
- presentation.

Extensions own:

- capabilities,
- behaviour,
- information.

Responsibilities remain intentionally separated.

Good Examples

Streaming Extension

Behaviour.



Playback Expression.



Hero Tile.



Presentation.

Users experience normal playback.

Book Provider

Behaviour.



Relationship Expression.



Relationship Tile.



Presentation.

Discovery behaves identically to native content.

Download Extension

Capability.



Download Expression.



Action Tile.



Presentation.

Interaction feels entirely native.

Anti-patterns

Custom Widgets

Extensions rendering independent UI.

Custom Tile Types

Extensions inventing new behavioural presentation primitives.

Theme Injection

Extensions modifying:

- Materials,
- Typography,
- Motion.

Runtime Ownership

Extensions bypassing the Composition Engine.

Extension Tile Model

[mermaid]
flowchart TD

```
Extension
Extension --> Behaviour
Behaviour --> Expression
Expression --> CompositionEngine
CompositionEngine --> TileFramework
TileFramework --> ResolvedTile
ResolvedTile --> Presentation
```

Extensions contribute behaviour.

The platform owns presentation.

Relationship To Future Chapters

The next chapter defines **Tile Orchestration**.

Extension Tiles explain:

How extensions participate in the Tile Framework.

Tile Orchestration explains:

How every Tile evolves together during runtime while preserving behavioural continuity.

Together they complete the behavioural integration architecture of the Tile Framework.

Summary

Extension Tiles ensure that the Mosaic ecosystem grows without fragmenting.

Extensions enrich:

- the Runtime World,
- behavioural understanding,
- user capability.

The Tile Framework transforms those contributions into presentation that feels entirely native.

Users should never think:

"This came from an extension."

They should simply feel that Mosaic naturally became more capable.

Review Status

Status

Draft

Next File

10-tile-orchestration.md

Tile Orchestration

Purpose

The Tile Framework transforms behavioural understanding into presentation.

However...

A real user experience is rarely the result of one Tile acting independently.

Hero Tiles.

Timeline Tiles.

Relationship Tiles.

Overlay Tiles.

All evolve together as the Runtime World changes.

Tile Orchestration ensures every Tile participates in one coherent behavioural experience.

Users should never perceive independent Tiles updating.

They should perceive one World naturally evolving.

Definition

Within MDS, **Tile Orchestration** is defined as:

The coordinated runtime evolution of every active Tile following behavioural change while preserving continuity, hierarchy and presentation coherence.

Tile Orchestration coordinates presentation.

It does not determine behaviour.

Behaviour remains the responsibility of the Composition Engine.

Philosophy

Traditional interfaces often behave like this.

[text]
Widget

↓

Updates

↓

Neighbour Updates

↓

Animation

↓

Finished

Each component behaves independently.

Mosaic intentionally behaves differently.

[text]

Behaviour

↓

Composition

↓

Tiles

↓

Presentation

↓

Understanding

Tiles evolve together because behaviour changed.

Not because rendering occurred.

Behaviour Is Authority

Every orchestration cycle begins with behaviour.

Examples.

[text]

Playback Started

Playback Paused

Focus Changed

Search Opened

Episode Completed

Bookmark Added

Rendering events never initiate Tile Orchestration.

Behaviour always possesses the highest authority.

Orchestration Pipeline

Every behavioural update follows the same conceptual sequence.

[text]

Behaviour

↓

Composition Engine

↓

Expression Resolution

↓

Tile Resolution

↓

Tile Orchestration

↓

Presentation

↓

Rendering

Tile Orchestration never bypasses earlier runtime stages.

Orchestration Responsibilities

Tile Orchestration coordinates:

- Tile activation
- Tile evolution
- Tile retirement
- Material synchronisation
- Motion sequencing
- Presentation continuity

It intentionally avoids:

- behavioural solving
- rendering
- platform implementation

Those responsibilities belong elsewhere.

Hero First

The Hero remains the behavioural centre.

When orchestration occurs:

[text]
Hero

↓

Primary

↓

Supporting

↓

Peripheral

↓

Environment

Tile evolution should always respect Runtime Hierarchy.

Group Coordination

Tile groups evolve together.

Example.

Playback.

[text]

Hero Group

↓

Timeline

↓

Actions

↓

Metadata

The entire group should communicate one behavioural transition.

Groups should never fragment into unrelated animations.

Material Coordination

Tile Orchestration synchronises Material behaviour.

Example.

Hero evolves.

↓

Hero Material updates.

↓

Supporting Acrylic responds.

↓

Atmosphere redistributes.

↓

Canvas settles.

Every material should appear to belong to one physical environment.

Typography Coordination

Typography should remain behaviourally stable.

Tile Orchestration should ensure:

- editorial hierarchy remains intact
- reading rhythm is preserved
- emphasis changes only when behaviour changes

Words should remain easier to follow than movement.

Motion Coordination

Tile Orchestration coordinates Motion.

Example.

[text]
Hero Tile

↓

Hero Motion

↓

Supporting Motion

↓

Environmental Motion

Motion should reinforce Tile relationships.

It should never create new behavioural meaning.

Incremental Orchestration

Most runtime updates affect only a small number of Tiles.

Example.

Playback progress.

↓

Timeline Tile evolves.

↓

Hero unchanged.

↓

Relationships unchanged.

Only affected Tiles should participate.

This preserves both performance and continuity.

Tile Dependencies

Future implementations may internally maintain Tile dependency graphs.

Conceptually.

[text]
Hero Tile

↓

Timeline Tile

↓

Metadata Tile

Dependencies improve scheduling.

They should never alter behavioural ordering.

Stable Identity

Whenever practical...

Tiles should evolve rather than be replaced.

Preferred.

[text]
Hero Tile

↓

Updated Hero Tile

Avoid.

[text]
Hero Removed

↓

Hero Created

Identity continuity significantly reduces cognitive effort.

Environmental Coordination

Environmental presentation should always respond last.

Example.

[text]
Behaviour

↓

Tiles

↓

Motion

↓

Atmosphere

↓

Environment Settles

The world should appear to respond naturally.

Not mechanically.

Adaptive Coordination

Adaptive behaviour should participate naturally.

Examples.

Orientation changes.

↓

Tile variants evolve.

Window resized.

↓

Groups adapt.

Accessibility enabled.

↓

Presentation refines.

The orchestration model remains identical.

Only presentation changes.

Failure Behaviour

If one Tile cannot update.

Preferred.

[text]

Tile Fallback

↓

Group Continues

Avoid.

[text]

Entire Composition Stops

Behavioural continuity should always remain the highest priority.

Deterministic Behaviour

Given identical:

- Runtime World
- Expressions
- Tile identities

Tile Orchestration should always produce identical runtime behaviour.

Determinism enables:

- replay
- debugging
- caching
- synchronisation

Every Mosaic client should therefore orchestrate Tiles identically.

Multi-Device Coordination

Every device follows the same orchestration model.

Desktop.

↓

Same Tile sequence.

Phone.

↓

Same Tile sequence.

Television.

↓

Same Tile sequence.

Presentation fidelity changes.

Behavioural orchestration does not.

Plugins

Extensions contribute:

- Expressions
- Behaviour
- Relationships

Plugins never orchestrate Tiles.

The Tile Framework owns:

- sequencing
- grouping
- presentation evolution

Every extension therefore automatically inherits one coherent presentation language.

Good Examples

Playback

Playback begins.

↓

Hero Group evolves.

↓

Timeline updates.

↓

Atmosphere settles.

↓

Presentation completes.

Everything feels like one behavioural transition.

Reading

Bookmark added.

↓

Reading Group updates.

↓

Typography remains stable.

↓

Reader continues naturally.

Search

Overlay Group appears.

↓

Interaction.

↓

Overlay retires.

↓

Previous Hero Group immediately resumes.

Continuity is preserved.

Anti-patterns

Independent Tiles

Every Tile animates separately.

Component Orchestration

Widgets coordinating runtime behaviour.

Platform Orchestration

Different clients inventing different sequencing.

Plugin Orchestration

Extensions scheduling their own presentation updates.

Tile Orchestration Model

[mermaid]
flowchart TD

```
Behaviour
Behaviour --> CompositionEngine
CompositionEngine --> Expressions
Expressions --> Tiles
Tiles --> TileOrchestration
TileOrchestration --> Presentation
Presentation --> Rendering
```

One behavioural event.

One coordinated presentation evolution.

Relationship To Future Chapters

The next chapter defines **Tile Framework Governance**.

Tile Orchestration explains:

How Tiles evolve together.

Governance explains:

How the Tile Framework continues evolving while preserving one behavioural presentation language across the lifetime of Mosaic.

Together they complete the runtime architecture of the Tile Framework.

Summary

Tile Orchestration is the presentation conductor of Mosaic.

It ensures that:

- Tiles evolve together,
- Materials remain coherent,
- Motion remains meaningful,
- Typography remains readable,
- the user's World remains continuous.

Users should never perceive individual Tiles updating.

They should simply feel that their World naturally continued.

Review Status

Status

Draft

Next File

11-governance.md

Tile Framework Governance

Purpose

The Tile Framework defines the reusable presentation language of Mosaic.

Unlike rendering technologies, which are expected to evolve rapidly, Tile identities become part of the long-term behavioural vocabulary of the platform.

Poor governance would gradually produce:

- duplicate Tile types,
- presentation fragmentation,
- inconsistent interaction,
- platform divergence,
- extension incompatibility.

This chapter defines how the Tile Framework should evolve while preserving one coherent presentation language.

Governance Philosophy

Tiles should evolve technically.

Their behavioural meaning should remain remarkably stable.

The objective is not preserving implementations.

It is preserving:

- behavioural consistency,
- presentation reuse,
- adaptive flexibility,
- runtime clarity.

Future UI technologies should strengthen the Tile Framework rather than replace it.

Tiles Are Architecture

Tiles are not widgets.

They are architectural presentation primitives.

Changing:

- Hero Tile,
- Timeline Tile,
- Relationship Tile,

changes how users perceive runtime behaviour.

Such changes should therefore receive architectural review rather than implementation review.

Stable Responsibilities

The following concepts should remain highly stable.

- Tile Philosophy
- Tile Taxonomy
- Expression Mapping
- Tile Lifecycle
- Adaptive Tiles
- Tile Interaction

These concepts define the public presentation language of Mosaic.

Evolvable Responsibilities

The following may evolve continuously.

- rendering implementations
- adaptive variants
- platform components
- caching strategies
- runtime optimisation
- presentation fidelity

Implementation may evolve.

Behavioural identity should remain stable.

Tile Ownership

Tile responsibilities are intentionally separated.

Layer	Owner
Tile Philosophy	Design Systems
Tile Taxonomy	Design Systems
Runtime Tile Resolution	Runtime Architecture
Component Library	Client Platform
Rendering	Platform Runtime

Ownership preserves presentation consistency while allowing implementation to improve independently.

Introducing New Tiles

Before introducing a new Tile ask:

Question One

Can an existing Tile already communicate this Expression?

Question Two

Is this genuinely a new behavioural presentation...

or simply another visual variation?

Question Three

Could adaptive behaviour solve this instead?

Question Four

Will this Tile remain meaningful after future rendering technologies change?

Question Five

Would another contributor naturally discover this Tile?

If uncertainty remains...

The proposal should continue evolving before implementation.

Tile Drift

Tile Drift occurs when:

- duplicate Tile identities appear,
- platform-specific Tiles emerge,
- extensions introduce presentation primitives,
- behaviour leaks into components,
- rendering influences runtime vocabulary.

Tile Drift weakens one of the strongest architectural separations within Mosaic.

It should therefore be treated as architectural debt.

Tile Debt

Examples include:

- duplicate Hero Tiles,
- platform variants becoming independent identities,
- widget-driven behaviour,
- presentation-owned interaction,
- undocumented Tile exceptions.

Tile Debt should be continuously reduced.

The Tile vocabulary should remain intentionally compact.

Expression Governance

Expressions and Tiles intentionally remain separate.

Expressions communicate:

- understanding.

Tiles communicate:

- presentation.

Neither layer should absorb responsibilities belonging to the other.

Maintaining this separation preserves long-term architectural flexibility.

Adaptive Governance

Adaptive behaviour should never create new Tile identities.

Desktop Hero.

↓

Hero Tile.

Phone Hero.

↓

Hero Tile.

Voice Hero.

↓

Hero Tile.

Adaptive behaviour projects.

It never redefines.

Accessibility Governance

Accessibility always possesses higher authority than presentation richness.

No Tile proposal should weaken:

- readability,
- interaction,
- behavioural clarity,
- continuity.

Accessibility refines Tiles.

It never changes their identity.

Plugin Governance

Extensions must never define:

- Tile families,
- adaptive behaviour,
- presentation hierarchy,
- interaction models.

Plugins contribute:

- Expressions,
- behaviours,
- relationships.

The Tile Framework owns presentation.

This guarantees every extension feels unmistakably Mosaic.

Review Questions

Every Tile proposal should answer:

- Does this strengthen behavioural understanding?
- Does this preserve reuse?
- Does this remain platform independent?
- Could an existing Tile already solve this?
- Would users still recognise Mosaic?
- Is this solving behaviour or implementation?

If the proposal exists primarily because a UI framework makes it convenient...

It should be reconsidered.

Behaviour always possesses higher authority than implementation.

Validation

Future tooling should automatically validate:

- Tile taxonomy usage
- Expression mapping
- adaptive variants
- runtime consistency
- extension compatibility
- platform parity

Validation should reinforce architectural review.

It should never replace behavioural reasoning.

Governance Workflow

[mermaid]
flowchart TD

```
Proposal
Proposal --> ExpressionReview
ExpressionReview --> ExistingTile["ExistingTile?"]
ExistingTile["ExistingTile?"] -->|Yes| Reuse
ExistingTile["ExistingTile?"] -->|No| ArchitectureReview
ArchitectureReview
ArchitectureReview --> AdaptiveReview
AdaptiveReview --> AccessibilityReview
AccessibilityReview --> Approval
Approval --> Implementation
```

Refinement should always be preferred over expanding the Tile vocabulary.

Success Criteria

The Tile Framework succeeds when:

- Expressions consistently resolve into reusable Tiles,
- adaptive behaviour remains predictable,
- rendering frameworks remain replaceable,
- contributors naturally think in Tiles rather than widgets,
- extensions integrate seamlessly,
- users experience one coherent presentation language.

Tiles should become invisible.

Only understanding should remain.

Architectural Decisions

ADR	Decision
ADR-163	Tiles are behavioural presentation primitives rather than UI components.
ADR-164	Expressions map deterministically into Tile identities.
ADR-165	Adaptive behaviour never changes Tile identity.
ADR-166	Runtime Tile Resolution owns presentation behaviour.
ADR-167	Extensions inherit the Tile Framework rather than extending it.

Review Status

Status

Draft

Next File

12-adrs.md

Architectural Decision Records

Purpose

The Architectural Decision Records (ADRs) contained within MDS-007 preserve the architectural reasoning behind the Mosaic Tile Framework.

Every previous specification established:

- Behaviour
- Runtime World
- Composition
- Expressions

MDS-007 establishes how those runtime concepts become reusable presentation primitives.

These ADRs explain why Mosaic deliberately separates:

- Expressions,
- Tiles,
- Components,
- Rendering,

into independent architectural layers.

Future contributors should understand these decisions before modifying the Tile Framework.

ADR Format

Every Mosaic ADR follows the standard structure.

[text]

ADR Number

Status

Context

Decision

Consequences

Alternatives Considered

Related Specifications

Each ADR documents one architectural decision.

ADR-168

Title

Introduce Tiles Between Expressions And Components

Status

Accepted

Context

Directly rendering Expressions tightly couples runtime behaviour to implementation technology.

Decision

Introduce Tiles as reusable presentation primitives positioned between Expressions and Components.

Consequences

Behaviour remains independent from rendering frameworks while presentation becomes reusable.

ADR-169

Title

Tiles Represent Behaviour Rather Than Widgets

Status

Accepted

Context

Traditional UI frameworks frequently name presentation primitives after implementation.

Decision

Tile identities communicate behavioural purpose rather than component type.

Consequences

Tile vocabulary remains stable even as rendering technologies evolve.

ADR-170

Title

Expression Mapping Must Remain Deterministic

Status

Accepted

Context

Platform-specific presentation decisions create inconsistent runtime behaviour.

Decision

Identical Expressions must always resolve into identical Tile identities.

Consequences

Caching, replay, testing and cross-platform consistency become significantly simpler.

ADR-171

Title

Adaptive Behaviour Never Changes Tile Identity

Status

Accepted

Context

Responsive interfaces frequently create separate presentation concepts for different devices.

Decision

Adaptive behaviour modifies presentation only.

Tile identity remains unchanged.

Consequences

One behavioural language survives across every device.

ADR-172

Title

Tile Lifecycle Preserves Identity

Status

Accepted

Context

Destroying and recreating presentation primitives weakens continuity.

Decision

Tiles evolve whenever practical instead of being replaced.

Consequences

Users perceive one continuous World rather than constantly changing interface objects.

ADR-173

Title

Tile Interaction Represents Behaviour

Status

Accepted

Context

Component-owned callbacks tightly couple interaction to implementation.

Decision

Tiles expose behavioural interaction intent.

Components merely implement interaction mechanisms.

Consequences

Interaction remains platform independent and behaviourally consistent.

ADR-174

Title

Runtime Tile Resolution Owns Presentation

Status

Accepted

Context

Allowing components to determine Materials, Typography or Motion fragments presentation.

Decision

Runtime Tile Resolution produces fully resolved Tiles before rendering begins.

Consequences

Components become extremely small implementation artefacts.

ADR-175

Title

Extensions Never Define Tiles

Status

Accepted

Context

Plugin-owned presentation primitives fragment behavioural consistency.

Decision

Extensions contribute Expressions.

The Tile Framework determines presentation.

Consequences

Community extensions automatically inherit future presentation improvements.

ADR-176

Title

Tile Orchestration Coordinates Presentation

Status

Accepted

Context

Independent Tile updates weaken behavioural continuity.

Decision

Tile evolution is orchestrated centrally.

Consequences

Users experience one coherent presentation rather than many unrelated interface updates.

ADR Relationships

[mermaid]
flowchart TD

ADR168["Tiles"]

ADR169["Behaviour"]

ADR170["Deterministic Mapping"]

ADR171["Adaptive Identity"]

ADR172["Lifecycle"]

ADR173["Interaction"]

ADR174["Runtime Resolution"]

ADR175["Extensions"]

ADR176["Orchestration"]

ADR168 --> ADR169

ADR169 --> ADR170

ADR170 --> ADR171

ADR171 --> ADR172

ADR172 --> ADR174

ADR173 --> ADR174

ADR174 --> ADR176

ADR174 --> ADR175

Together these decisions establish Tiles as the stable presentation language bridging runtime understanding and visual implementation.

Future ADRs

Future Tile Framework ADRs are expected to formalise:

- AI-generated Tile Variants
- Spatial Tile Projection
- Ambient Tile Groups
- Multi-user Shared Tiles
- Predictive Tile Resolution
- Streaming Tile Pipelines
- Adaptive Accessibility Tiles
- Runtime Tile Virtualisation

These intentionally remain outside the scope of MDS-007 Version 0.1.

ADR Governance

Tile Framework ADRs should change only when:

- behavioural architecture evolves,

- runtime presentation requires refinement,
- accessibility research identifies deficiencies,
- the Mosaic Design Language itself changes.

Rendering technology alone should never justify architectural changes.

Tiles should remain recognisably Mosaic regardless of implementation.

Summary

The ADRs contained within MDS-007 define the presentation identity of Mosaic.

Expressions communicate understanding.

Tiles communicate presentation.

Components implement rendering.

Maintaining these boundaries allows Mosaic to evolve for decades without losing its behavioural language.

Review Status

Status

Draft

Next File

13-contributor-guidance.md

Contributor Guidance

Purpose

The Tile Framework is the bridge between runtime understanding and visual presentation.

Every contributor working on:

- runtime,
- frontend,
- extensions,
- components,
- interaction,
- presentation,

will eventually interact with Tiles.

This guidance exists to ensure contributors think in behavioural presentation rather than implementation.

When contributors naturally think in Tiles instead of widgets, the Tile Framework has achieved its purpose.

Think In Expressions

Never begin with:

"Which component should I use?"

Instead ask:

"Which Expression am I communicating?"

Good.

[text]
Timeline

↓

Timeline Tile

Poor.

[text]
Progress Bar

↓

Component

Expressions always precede Tiles.

Think In Tiles

After identifying the Expression ask:

"Which Tile best communicates this behaviour?"

Not:

Which widget?

Not:

Which layout?

Tiles represent presentation intent.

Components merely implement that intent.

Behaviour Before Presentation

Every Tile exists because behaviour exists.

Preferred.

[text]
Behaviour

↓

Expression

↓

Tile

↓

Component

Avoid.

[text]
Widget

↓

Behaviour

Presentation should always remain downstream from runtime understanding.

Preserve Tile Identity

Tiles should remain behaviourally stable.

Example.

Hero Tile

may appear differently on:

- desktop,

- phone,
- television,
- voice.

It remains:

Hero Tile

Do not invent new Tile identities simply because presentation differs.

Reuse Existing Tiles

Before creating a new Tile ask:

Can an existing Tile communicate this?

Most new presentation requests should refine existing Tiles rather than expand the Tile vocabulary.

Behavioural simplicity is significantly more valuable than presentation variety.

Components Render

Components should never own behaviour.

Good.

[text]
Hero Tile

↓

Hero Component

Poor.

[text]
Hero Component

↓

Hero Behaviour

Components are implementation.

Tiles communicate behaviour.

Let Runtime Resolve

Applications should never determine:

- Materials,
- Typography,
- Motion,
- Interaction,

- Adaptive behaviour.

Components consume resolved Tiles.

The Runtime Tile Resolver owns presentation logic.

Respect Adaptive Behaviour

Do not create:

- Mobile Hero Tile
- TV Hero Tile
- Desktop Hero Tile

Instead create:

Hero Tile

↓

Adaptive Projection

Devices change.

Behaviour should not.

Preserve Continuity

Whenever practical...

Tiles should evolve.

Not be replaced.

Preferred.

[text]
Hero Tile

↓

Updated Hero Tile

Avoid.

[text]
Destroy

↓

Create

Users should perceive continuous presentation.

Respect Runtime Hierarchy

Tiles never determine importance.

Hierarchy originates from:

- Behaviour,
- Composition,
- Runtime Hierarchy.

Tiles communicate that hierarchy.

They never redefine it.

Plugins Enrich Behaviour

Extensions contribute:

- behaviour,
- information,
- relationships.

They never contribute:

- Tiles,
- layouts,
- widgets,
- Materials.

Every extension should disappear naturally into Mosaic.

Accessibility

Accessibility should adapt presentation.

It should never create new Tile identities.

Examples.

Large text.

↓

Hero Tile.

Reduced motion.

↓

Hero Tile.

High contrast.

↓

Hero Tile.

Behaviour remains unchanged.

Platform Independence

Before implementing any Tile ask:

Would this still communicate the same behaviour on:

- Web?
- Flutter?
- Television?
- Voice?
- Future devices?

Presentation may evolve.

Behaviour should remain identical.

Performance

Performance optimisation should preserve Tile identity.

Preferred.

[text]
Same Tile

↓

Simpler Rendering

Avoid.

[text]
Different Tile

↓

Higher Performance

Optimise implementation.

Never behavioural presentation.

Common Mistakes

Avoid the following.

Widget Thinking

Beginning with components instead of Expressions.

Platform Thinking

Creating platform-specific Tile identities.

Layout Thinking

Allowing geometry to redefine behaviour.

Plugin Presentation

Extensions creating their own UI language.

Component Behaviour

Widgets mutating the Runtime World.

Duplicate Tiles

Creating new Tiles for existing behavioural concepts.

Tile Review Questions

Before implementing a Tile ask:

- Which Expression does this communicate?
- Does an existing Tile already solve this?
- Does this preserve behavioural identity?
- Does adaptive behaviour remain consistent?
- Would this still feel like Mosaic on every platform?
- Could the renderer change without affecting this design?

If uncertainty remains...

Return to the Expression before implementation.

Tile Checklist

Every Tile implementation should satisfy the following.

- ☐ Expression is clearly identified.
- ☐ Existing Tile reused where possible.
- ☐ Behaviour remains platform independent.
- ☐ Runtime Hierarchy is respected.
- ☐ Adaptive behaviour preserves identity.
- ☐ Accessibility is maintained.
- ☐ Components remain implementation only.
- ☐ Extensions cannot bypass the Tile Framework.

Final Guidance

The Tile Framework should eventually disappear from conscious thought.

Contributors should stop asking:

"Which widget should I build?"

and instinctively begin asking:

"Which behavioural idea am I presenting?"

When every contributor naturally thinks in Expressions and Tiles rather than components, Mosaic gains something rare.

A presentation layer that can evolve for decades while the behavioural language remains unchanged.

That separation is the defining strength of the Mosaic Tile Framework.

Review Status

Status

Draft

Next File

glossary.md

Glossary

Purpose

This glossary defines the terminology introduced by **MDS-007 — Tile Framework**.

Unlike previous glossaries, these definitions describe the presentation abstraction layer that exists between runtime understanding and rendered components.

These definitions should remain stable regardless of:

- rendering technology,
- UI framework,
- platform,
- device.

Future specifications should reuse these definitions consistently.

A

Adaptive Tile

A Tile capable of altering its physical presentation while preserving behavioural identity.

Adaptive Tiles change:

- layout,
- density,
- interaction,
- presentation.

They never change:

- behaviour,
- meaning,
- runtime identity.

C

Collection Tile

A Tile representing an organised behavioural group of related content.

Examples include:

- Continue Watching,
- Library,

- Recently Added,
- Favourites.

Collections communicate organisation rather than hierarchy.

Content Tile

A Tile representing an individual content object.

Examples include:

- film,
- episode,
- chapter,
- album,
- book.

Content Tiles become important through Runtime Hierarchy rather than Tile identity.

E

Expression Mapping

The deterministic process through which Expressions become Tile identities.

Expression Mapping preserves behavioural meaning while remaining independent from rendering technology.

Extension Tile

A standard Mosaic Tile produced from an extension-provided Expression.

Extensions never define Tile behaviour directly.

The Tile Framework remains responsible for presentation.

H

Hero Tile

The Tile representing the current behavioural centre of the user's World.

Hero Tiles inherit:

- Hero Material,
- Hero Typography,
- Hero Motion.

Only one Hero Tile normally exists within a Composition.

M

Metadata Tile

A Tile communicating supporting information.

Examples include:

- runtime,
- codec,
- language,
- release year.

Metadata Tiles remain behaviourally quiet.

O

Overlay Tile

A temporary Tile supporting interaction.

Examples include:

- search,
- playback controls,
- command palettes,
- menus.

Overlay Tiles inherit Overlay Materials and Overlay Motion.

P

Presentation Primitive

A reusable behavioural presentation object independent from rendering implementation.

Within Mosaic, Tiles are presentation primitives.

Components are not.

R

Relationship Tile

A Tile communicating behavioural relationships.

Examples include:

- recommendations,
- cast,
- franchise,
- author.

Relationship Tiles encourage exploration while preserving the user's current World.

Runtime Tile Resolution

The deterministic process that transforms behavioural Tile identities into fully resolved presentation objects.

Runtime Tile Resolution evaluates:

- Materials,
- Typography,
- Motion,
- Accessibility,
- Adaptive behaviour

before components are created.

T

Tile

A reusable behavioural presentation primitive representing one solved Expression.

Tiles communicate presentation.

They do not communicate rendering implementation.

Tile Composition

The behavioural grouping of multiple Tiles into coherent presentation structures.

Tile Composition follows behavioural relationships rather than layout.

Tile Framework

The architectural layer positioned between Expressions and Components.

The Tile Framework transforms runtime understanding into reusable presentation primitives.

Tile Identity

The stable behavioural identity assigned to every Tile.

Examples include:

- Hero Tile,
- Timeline Tile,
- Metadata Tile.

Tile identity remains stable across:

- devices,
- platforms,
- rendering technologies.

Tile Lifecycle

The behavioural evolution of a Tile through:

- resolution,
- activation,
- evolution,
- retirement,
- release.

Lifecycle follows behaviour.

Not rendering.

Tile Orchestration

The coordinated runtime evolution of every active Tile.

Tile Orchestration ensures presentation evolves as one coherent behavioural experience.

Tile Taxonomy

The canonical behavioural vocabulary of Tile families used throughout Mosaic.

Tile Taxonomy intentionally remains:

- compact,
- reusable,
- presentation independent.

Timeline Tile

A Tile communicating temporal progression.

Examples include:

- playback progress,
- reading progress,

- listening position,
- queue position.

Timeline Tiles communicate continuity rather than raw numerical progress.

U

Utility Tile

A Tile communicating peripheral system information.

Examples include:

- diagnostics,
- synchronisation,
- storage,
- updates.

Utility Tiles should remain behaviourally secondary.

Cross References

Specification	Primary Concepts
MDL-001 Vision	Companion
MDL-002 Principles	Behaviour Before Interface
MDL-003 Mental Model	World
MDL-004 Interaction Model	Behaviour
MDL-005 Composition Model	Expressions
MDS-006 Composition Engine	Runtime World, Expressions, Presentation Model

Terminology Rules

Future contributors should:

- describe Expressions before Tiles,
- describe Tiles before Components,
- distinguish presentation from rendering,
- distinguish behavioural identity from layout,
- avoid implementation-specific terminology within architectural documentation.

Tile terminology should remain independent from UI frameworks.

Review Status

Status

Draft

Next File

references.md

References

Purpose

This document records the architectural influences and conceptual foundations that informed **MDS-007 — Tile Framework**.

Unlike implementation documentation, these references explain *why* the Tile Framework exists rather than prescribing UI frameworks or component libraries.

The Tile Framework intentionally combines ideas from:

- behavioural presentation,
- information architecture,
- runtime composition,
- adaptive interfaces,
- design systems,
- reusable presentation primitives,

into one architectural layer that separates understanding from implementation.

Reading Order

Contributors should approach references in the following order.

1. MDL Specifications 2. Design Token Architecture 3. Colour System 4. Material System 5. Typography System 6. Motion System 7. Composition Engine 8. Tile Framework 9. Component Library

The Mosaic Design Language remains the architectural authority.

Rendering technologies merely implement its decisions.

Internal References

MDL-001 — Vision

Provides:

- Companion philosophy
- Entertainment-first thinking
- Calm interaction

Tiles should communicate the Companion's understanding rather than expose implementation details.

MDL-002 — Principles

Provides:

- Behaviour Before Interface
- Content Leads
- Calm Interfaces
- Every Feature Earns Its Place

Tiles exist because behaviour already exists.

Presentation should never lead behaviour.

MDL-003 — Mental Model

Provides:

- World
- Focus
- Context
- Relationships

Tiles communicate the current World.

They never define it.

MDL-004 — Interaction Model

Provides:

- Behaviour
- Continuity
- Runtime evolution

Tile interaction is a behavioural consequence rather than a component capability.

MDL-005 — Composition Model

Provides:

- Hero
- Expressions
- Hierarchy
- Priority

The Tile Framework is the presentation implementation of the Composition Model.

MDS-001 — Design Token Architecture

Provides:

- Runtime Resolution

- Semantic Tokens

Runtime Tile Resolution extends the deterministic resolution architecture introduced by MDS-001.

MDS-002 — Colour System

Provides:

- Material intent
- Runtime Atmosphere
- Semantic Colour

Tiles inherit Colour behaviour through runtime resolution.

They never define colours independently.

MDS-003 — Material System

Provides:

- Hero Material
- Acrylic
- Overlay Material
- Runtime Material Resolution

Tiles inherit Material behaviour.

They do not create Material behaviour.

MDS-004 — Typography System

Provides:

- Editorial Hierarchy
- Runtime Typography
- Reading Rhythm

Tiles communicate editorial intent.

Typography renders it.

MDS-005 — Motion System

Provides:

- Behavioural Motion
- Material Motion
- Runtime Motion Resolution

Tiles inherit Motion behaviour.

They never define animation independently.

MDS-006 — Composition Engine

Provides:

- Runtime World
- Composition Solver
- Expressions
- Runtime Hierarchy
- Presentation Model

The Tile Framework exists entirely because the Composition Engine produces Expressions rather than components.

Future Specifications

The following specification directly depends upon MDS-007.

- MDS-008 Component Library

The Component Library renders Tiles.

It never replaces the Tile Framework.

Behavioural Presentation

The Tile Framework intentionally treats presentation as behavioural communication rather than widget composition.

Conceptually.

[text]
Behaviour

↓

Expression

↓

Tile

↓

Component

↓

Rendering

Behaviour remains the architectural authority.

Presentation communicates it.

Information Architecture

Tiles are strongly influenced by information architecture.

The framework intentionally represents:

- concepts,
- relationships,
- actions,
- hierarchy,

rather than visual controls.

Understanding therefore survives implementation changes.

Adaptive Interfaces

The Tile Framework assumes presentation changes continuously.

Examples include:

- screen size
- orientation
- viewing distance
- accessibility
- future devices

Adaptive behaviour therefore belongs to Tiles rather than components.

Runtime Resolution

The Tile Framework intentionally continues the runtime resolution model established throughout MDS.

Conceptually.

[text]
Expression

↓

Tile

↓

Runtime Resolution

↓

Presentation

↓

Rendering

Every layer performs one architectural responsibility.

Platform Independence

The Tile Framework intentionally separates:

[text]
Behaviour

↓

Presentation

↓

Rendering

Every client therefore shares identical behavioural presentation while remaining free to implement:

- Flutter
- React
- SwiftUI
- Compose
- future rendering technologies

without changing architectural behaviour.

Mosaic-Specific Influences

The Tile Framework emerged directly from founder exploration.

Major architectural discoveries included:

- Components are implementation, not architecture.
- Expressions should never render directly.
- Tiles provide the missing behavioural presentation layer.
- Adaptive behaviour should preserve Tile identity.
- Extensions should inherit presentation rather than define it.

Together these discoveries define the presentation architecture unique to Mosaic.

Relationship To The Companion

Tiles represent the visible body language of the Companion.

Conceptually.

[text]
world

↓

Expressions

↓

Tiles

↓

Presentation

↓

User

The Companion understands first.

Tiles communicate that understanding.

Components merely make it visible.

Normative References

Required reading before contributing to MDS-007.

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model
- MDS-001 Design Token Architecture
- MDS-002 Colour System
- MDS-003 Material System
- MDS-004 Typography System
- MDS-005 Motion System
- MDS-006 Composition Engine

Together these specifications define the conceptual foundation of the Tile Framework.

Informative References

Future contributors may also wish to review:

- MDS-008 Component Library

The Component Library explains how Tiles become concrete UI implementations while preserving every behavioural contract established by the Tile Framework.

Living Document

This reference list should remain intentionally concise.

References should only be introduced when they materially influence:

- behavioural presentation,
- runtime resolution,
- adaptive Tiles,
- implementation boundaries.

The objective is to preserve architectural reasoning rather than catalogue UI frameworks.

Completion

This concludes **MDS-007 — Tile Framework**.

The next specification in the Mosaic Design System is:

MDS-008 — Component Library

Where MDS-007 defines **how behavioural presentation is represented**, MDS-008 defines **how that presentation is physically implemented**.

It formalises:

- Component philosophy
- Component taxonomy
- Component contracts
- Component lifecycle
- Rendering architecture
- Platform implementations
- Accessibility contracts
- Component composition
- Runtime rendering

MDS-008 intentionally becomes the thinnest layer in the entire architecture.

By the time Components are created, every important decision has already been made.

Components simply render the user's World.